# A PROOF OF THEOREM 5.1

Proof. The proof follows from the definition of the `NextOffset` link. When `Version_i` is created at time $t_{i_{start}}$, the system links it to `Version_j`, the latest active version of the adjacent entity. This single operation implies two conditions:

**Temporal Order:** The creation of `Version_i` is causally dependent on the prior existence of `Version_j`. Thus, their creation times must be strictly ordered: $t_{i_{start}} > t_{j_{start}}$.

**Lifecycle Overlap:** For `Version_j` to be the *latest* version at time $t_{i_{start}}$, it must be active. This requires its lifecycle $[t_{j_{start}}, t_{j_{end}})$ to contain $t_{i_{start}}$, which means $t_{j_{end}} > t_{i_{start}}$.

Combining these results gives $t_{j_{start}} < t_{i_{start}} < t_{j_{end}}$. The inclusion of $t_{i_{start}}$ (the start of the first interval) within the second interval $[t_{j_{start}}, t_{j_{end}})$ directly proves a non-empty intersection. Both parts of the theorem hold. □

# B PROOF OF THEOREM 5.2

Proof. For this analysis, we establish an upper bound on the insertion failure probability. We simplify the model by considering only single-step hops; the actual failure probability is even lower due to the availability of multi-step chain hopping, which provides more opportunities to find an empty slot. Let $N$ be the total number of items currently in the hash table, $M$ be the maximum number of elements each hash bucket can contain, and $H$ be the `MaxHopDistance`. An insertion fails if an item hashes to a bucket and no empty slot can be found within its neighborhood of size $H$. We can bound the probability of this event by considering a more constrained scenario: the probability that a new item hashes to a specific bucket that is already the destination for $H$ existing items. Under the assumption of uniform hashing, the probability that a specific bucket is the destination for exactly $H$ of the $N$ items follows a binomial distribution:

$$P\{\text{H items in bucket}\} = \binom{N}{H}\left(\frac{1}{M}\right)^H\left(1-\frac{1}{M}\right)^{N-H} \tag{4}$$

$$= \frac{N!}{H!(N-H)!}\frac{(M-1)^{N-H}}{M^N} \tag{5}$$

$$= \frac{1}{H!}\cdot\frac{N!}{M^H(N-H)!}\cdot\left(\frac{M-1}{M}\right)^{N-H} \tag{6}$$

$$\leq \frac{1}{H!} \tag{7}$$

Here, equations (4) through (6) are equivalent formulations. The inequality in (7) holds because for a non-full table ($N + H < M$). An insertion failure requires that a new item hashes to such a "full" bucket. The probability of a new item hashing to any specific bucket is $1/M$. Therefore, the total probability of an insertion failure is bounded by:

$$P\{\text{Failure}\} = P\{\text{H items in bucket}\}\cdot P\{\text{Insert in this bucket}\} \tag{8}$$

$$\leq \frac{1}{H!}\cdot\frac{1}{M} \tag{9}$$

$$= \frac{1}{M\cdot H!} \tag{10}$$

Consequently, under uniform hashing, the probability of needing to resize the `Hopscotch Hash Table` is exceedingly small. □

**Table 3: Temporal Queries of T-mgBench**

| | |
|---|---|
| Q1 | Match (n: User {id:xx}) FOR TT AS OF $t$ RETURN n |
| Q2 | Match (n: User {id:xx}) FOR TT From $t_1$ to $t_2$ RETURN n |
| Q3 | Match (n: User {id:xx})-[e]->(m) FOR TT AS OF $t$ RETURN n,e,m |
| Q4 | Match (n: User {id:xx})-[e]->(m) FOR TT From $t_1$ to $t_2$ RETURN n,e,m |

**Table 4: Temporal Queries of T-LDBC**

| | |
|---|---|
| IS1 | MATCH (n:Person {id:xx})-[:IS_LOCATED_IN]-(p:Place)<br>FOR TT AS OF t<br>RETURN<br>n.firstName AS firstName, n.lastName AS lastName,<br>n.birthday AS birthday, n.locationIP AS locationIp,<br>n.browserUsed AS browserUsed, n.gender AS gender,<br>n.creationDate AS creationDate, p.id AS cityId; |
| IS2 | MATCH (:Person {id:xx})<-[:HAS_CREATOR]-(m)-[:REPLY_OF*0..]->(p:Post)<br>MATCH (p)-[:HAS_CREATOR]->(c)<br>FOR TT AS OF t<br>RETURN<br>m.id as messageId,<br>CASE m.content is not null<br>WHEN true<br>THEN m.content<br>ELSE m.imageFile<br>END AS messageContent,<br>m.creationDate AS messageCreationDate, p.id AS originalPostId,<br>c.id AS originalPostAuthorId, c.firstName as originalPostAuthorFirstName,<br>c.lastName as originalPostAuthorLastName<br>ORDER BY messageCreationDate DESC<br>LIMIT 10; |
| IS3 | MATCH (n:Person {id:xx})-[r:KNOWS]-(friend)<br>FOR TT AS OF t<br>RETURN<br>friend.id AS personId, friend.firstName AS firstName,<br>friend.lastName AS lastName,<br>r.creationDate AS friendshipCreationDate<br>ORDER BY friendshipCreationDate DESC, toInteger(personId) ASC; |
| IS4 | MATCH (m:Message {id:xx})<br>FOR TT AS OF t<br>RETURN<br>CASE exists(m.content)<br>WHEN true THEN m.content<br>ELSE m.imageFile<br>END AS messageContent, m.creationDate as messageCreationDate; |
| IS5 | MATCH (m:Message {id:xx})-[:HAS_CREATOR]->(p:Person)<br>FOR TT AS OF t<br>RETURN<br>p.id AS personId, p.firstName AS firstName, p.lastName AS lastName; |
| IS6 | MATCH (m:Message {id:xx})-[:REPLY_OF*0..]->(p:Post)<br><-[:CONTAINER_OF]-(f:Forum)-[:HAS_MODERATOR]->(mod:Person)<br>FOR TT AS OF t<br>RETURN<br>f.id AS forumId, f.title AS forumTitle, mod.id AS moderatorId,<br>mod.firstName AS moderatorFirstName, mod.lastName AS moderatorLastName; |
| IS7 | MATCH (m:Message {id:xx})<-[:REPLY_OF]-(c:Comment)-[:HAS_CREATOR]->(p:Person)<br>OPTIONAL MATCH (m)-[:HAS_CREATOR]->(a:Person)-[r:KNOWS]-(p)<br>FOR TT AS OF t<br>RETURN<br>c.id AS commentId, c.content AS commentContent,<br>c.creationDate AS commentCreationDate, p.id AS replyAuthorId,<br>p.firstName AS replyAuthorFirstName, p.lastName AS replyAuthorLastName,<br>CASE r<br>WHEN null THEN false<br>ELSE true END AS replyAuthorKnowsOriginalMessageAuthor<br>ORDER BY commentCreationDate DESC, toInteger(replyAuthorId) ASC; |

# C TEMPORAL QUERIES OF T-MGBENCH WORKLOAD

We list the temporal queries of the T-mgBench workload used in our experiment in Table 3. The values of the vertex id are sampled from the respective datasets on which the queries are evaluated.

# D TEMPORAL QUERIES OF T-LDBC WORKLOAD

We list the temporal queries of the T-LDBC workload used in our experiment in Table 4. The values of the vertex id are sampled from the respective datasets on which the queries are evaluated.